

Repulsive-surface optimization for self-intersection-free triangle meshes, implemented in C++/OpenMP with Barnes-Hut acceleration, Sobolev-style preconditioning, dynamic remeshing, a genus-0-through-genus-5 canonical embedding gallery, and a 30-frame sphere-through-tube interpolation demo.

Team: Atharv Sampath Michael Pham Daniel Liu Elen Pappan

[Submission Website](#) [Proposal](#) [Milestone](#) [Final Video](#) [Final Slides](#) [GitHub repo](#)

01: Abstract

Our original proposal aimed to implement *Repulsive Shells*, a recent 2024 paper about smoothly blending 3D shapes without them passing through themselves (self-intersecting). However, we realized that building the full complex mathematical solver required for this was too large for the timeframe of a class project. Therefore, our final project focuses on building the foundational system it relies on, called *Repulsive Surfaces*, plus one focused graph-manifold interpolation demo from the shell pipeline. We successfully implemented an algorithm that lowers tangent-point energy on 3D meshes, tested it on shapes with 0 to 5 holes (genus 0 through 5), and used the same repulsive potential to drive a 30-frame sphere-through-tube sequence.

Scope changed after the milestone: GPU acceleration (CUDA) and the complete Repulsive Shells system became stretch goals. We implemented enough of the shell/path machinery to ship the ball-through-tube demo, but our strongest validated result is the Repulsive Surfaces energy-minimization pipeline and gallery.

02: Technical Approach

Our system optimizes 3D meshes by minimizing a specific energy metric that heavily penalizes parts of the surface that get too close to intersecting. Because calculating this repulsive force between every single pair of triangles on a mesh is far too slow, we sped up the process using hierarchical data structures to group distant triangles together. Our final results show how messy or high-energy starting shapes move toward embedded, low-energy final surfaces without requiring explicit continuous collision detection.

To make this work efficiently and stably, our optimization pipeline follows four main steps:

1. Geometry Processing	We start by loading the 3D mesh and calculating basic geometric properties for each triangle face, such as its center point, orientation (normal), and area. These foundational properties are necessary to figure out exactly how the mesh should move to reduce the objective.
2. Fast Energy Calculation	Instead of a slow calculation that checks every triangle against every other triangle ($O(n^2)$ complexity), we use a Barnes-Hut approximation paired with Bounding Volume Hierarchies (BVH) and block-cluster trees. This allows us to group distant parts of the surface together and estimate their repulsive forces, saving a massive amount of computation time while still preventing self-intersections.
3. Smooth Descent	Simply moving the mesh's vertices based on basic forces can lead to jagged, unstable shapes. To fix this, we use advanced mathematical techniques (specifically, Sobolev-style preconditioning) to smartly guide the shape's movement. This ensures the mesh flows smoothly into a lower-energy embedded state over time.
4. Dynamic Remeshing	As the surface changes during energy minimization, the triangles making up the mesh can become highly distorted. To fix this, we dynamically "remesh" the surface as it updates, automatically splitting edges that get too long, collapsing ones that get too short, and smoothing out the triangles. This keeps the 3D model clean, uniform, and stable throughout the optimization.
Path demo	For the ball-through-tube example, we used the graph-manifold path energy from Repulsive Shells with shell elasticity, self TPE, a fixed-surface TPE barrier for the tube wall, and a small signed-distance guard. The trust-region solve optimized the interior frames of a 30-frame path while keeping the endpoints fixed.

Lessons learned

One of our biggest takeaways was underestimating the compute power required to iterate quickly; slow optimization runs made debugging and resolving issues a major bottleneck. Additionally, our main challenges were subtle numerical modeling mistakes rather than syntax bugs, such as biased energy aggregates, OpenMP summation non-

determinism changing optimizer paths, and remeshing altering resolution targets. Ultimately, we learned that research graphics code requires not only sufficient compute for rapid testing, but also exact baselines and strict regression tests around anything that changes the objective.

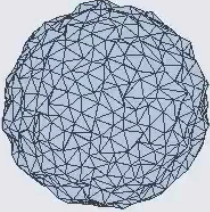
03: Results

We ran six gallery optimizations with `OMP_NUM_THREADS=4`, 500 iterations, dynamic remeshing every 10 iterations, and frame dumps at every iteration. Outputs are stored under `out/gallery_genusN`.

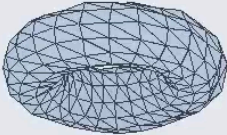
Repulsive Surfaces canonical embedding descent

frame 0000

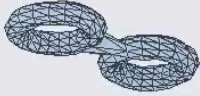
genus 0



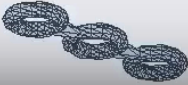
genus 1



genus 2



genus 3



▶ 0:00 / 0:16

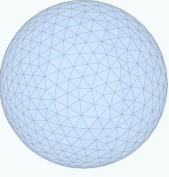
genus 4



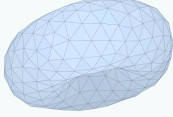
genus 5



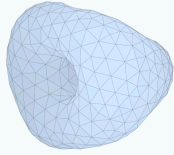
⏮ ⏪ ⏩ ⏭ ⋮



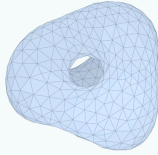
Genus 0. Sphere-like minimizer.



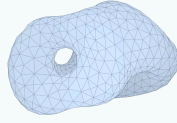
Genus 1. Torus of revolution.



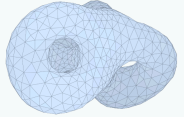
Genus 2. Multi-handle embedded state.



Genus 3. Higher-genus spread-out state.

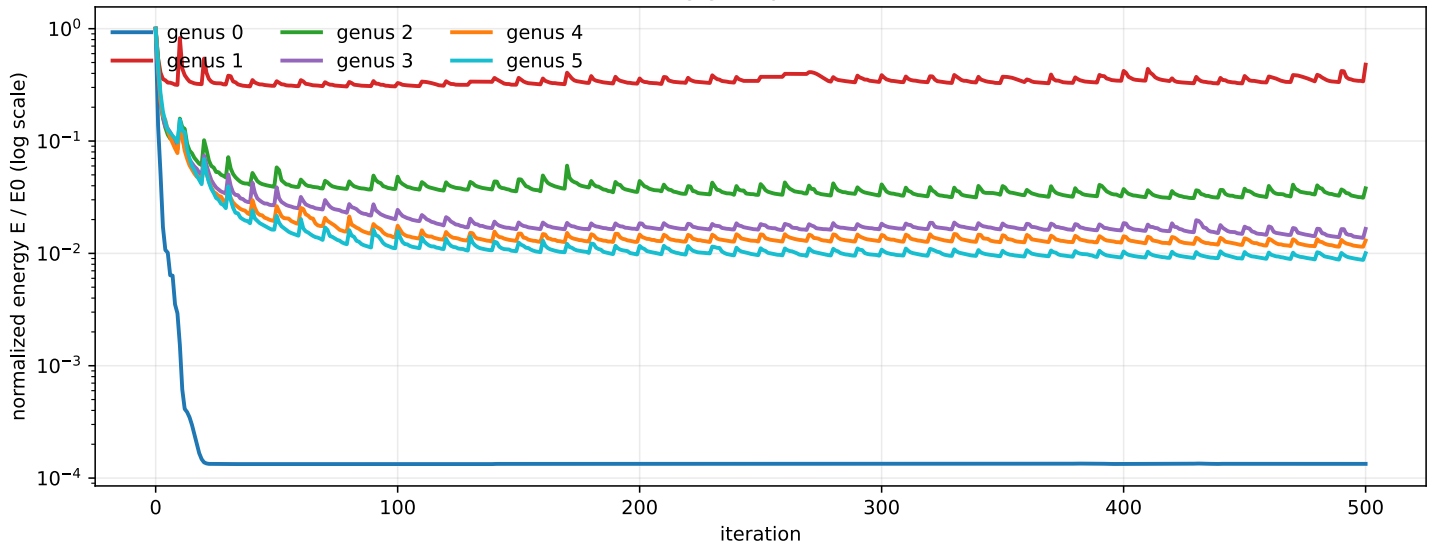


Genus 4. Bounded-remesh result.



Genus 5. Largest shipped gallery mesh.

Canonical embedding gallery: normalized TPE descent



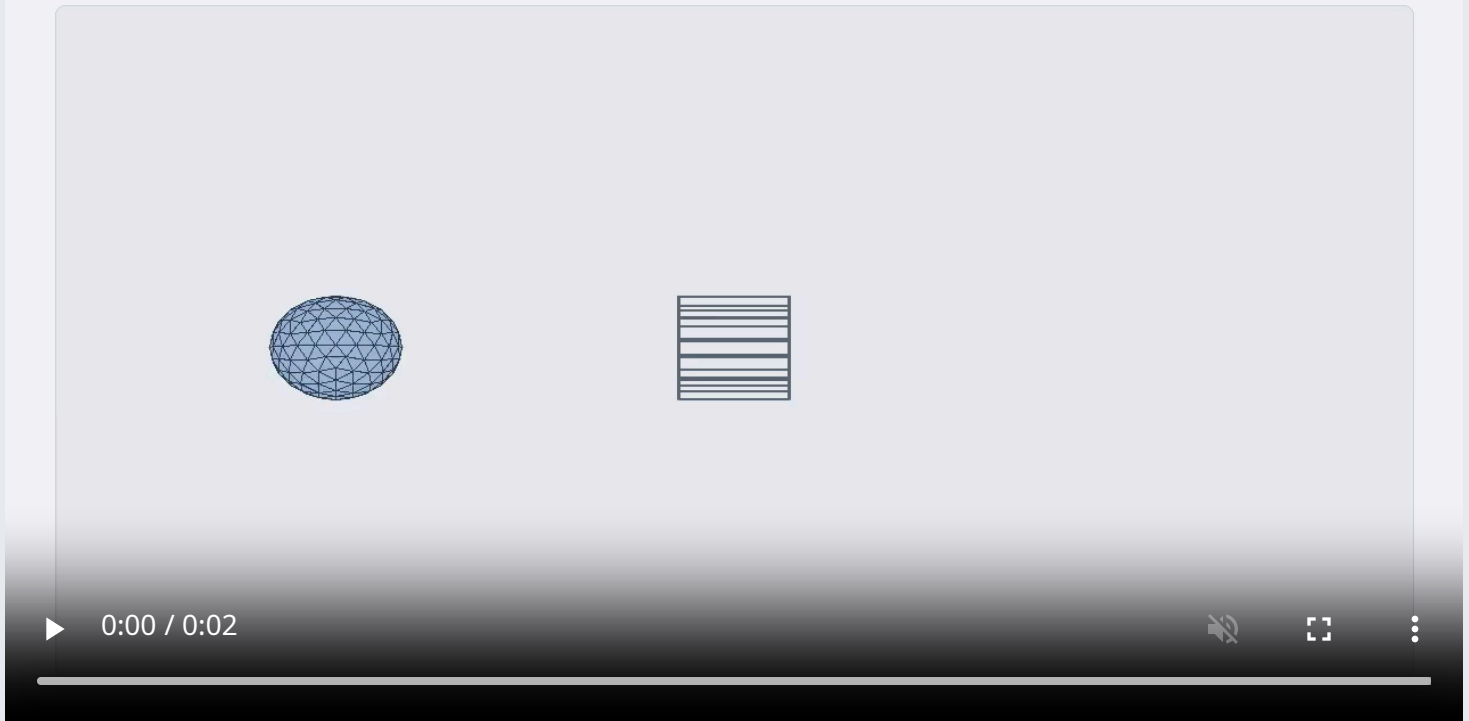
Normalized potential descent. The same runs plotted as E / E_0 show the relative decrease in tangent-point energy for each genus.

genus	start E	final E	drop	faces
0	231948	31.0357	7473.6x	1280 -> 1172
1	1909.6	913.572	2.1x	576 -> 528
2	43672.8	1656.33	26.4x	720 -> 828
3	142481	2353.83	60.5x	1080 -> 1242
4	346503	4505.47	76.9x	1440 -> 1656
5	701006	7038.95	99.6x	1800 -> 2070

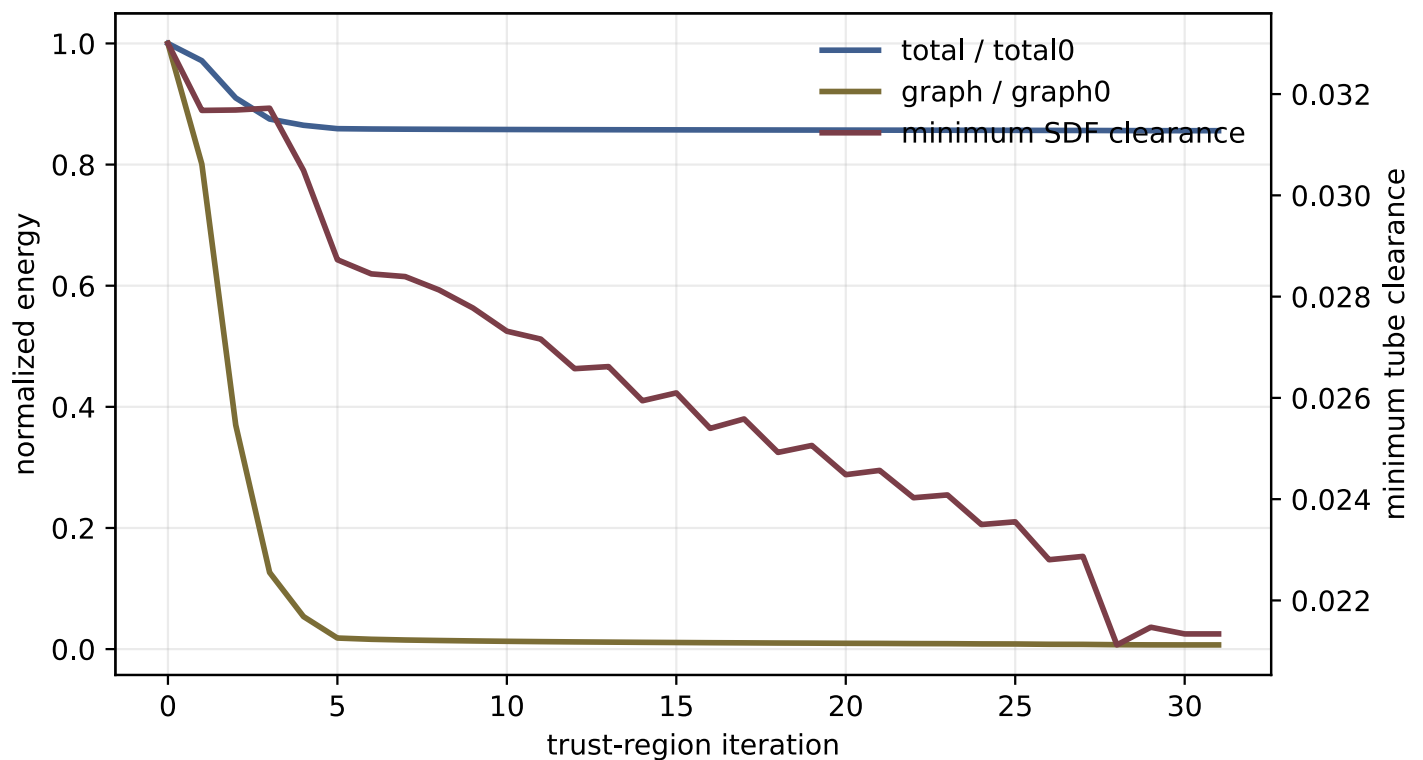
Genus 0 and 1 land on the expected round sphere and torus. The higher-genus examples are harder because the TPE landscape has many local minima; our shipped results are embedded, low-energy states rather than exact symmetry matches to every entry in the Repulsive Surfaces gallery.

Sphere-through-tube interpolation

We also completed a focused Repulsive Shells-style path demo: a sphere starts outside a hollow tube, enters the narrow channel, and exits the other side over 30 saved frames. The solve used shell energy for temporal shape regularity, TPE for self-repulsion, a fixed tube-surface TPE barrier, and a small SDF guard so coarse trial steps that entered the wall were rejected.



Ball-through-tube clip. Rendered from the 30 OBJ frames in `out/ball_tube_interp`. The tube is shown as a wire obstacle so the compressed sphere remains visible in the channel.



The graph repulsion term dropped from 0.0748 to 0.00052 over 30 accepted trust-region steps. The path stayed feasible throughout; the minimum signed tube clearance in the final sequence was 0.0213.

04: References

- Yu, Brakensiek, Schumacher, Crane (2021). *Repulsive Surfaces*. ACM SIGGRAPH Asia.

- Sassen, Schumacher, Rumpf, Crane (2024). *Repulsive Shells*. ACM SIGGRAPH.
- Henrik Schumacher's Repulsor reference implementation, used for comparison but not vendored into our project.
- C++17, CMake, Eigen, OpenMesh, OpenMP, NanoGUI/OpenGL, Polyscope, and Python/Matplotlib for report plots.

05: Contributions

- **Atharv Sampath:** Project scoping, paper/reference comparison, validation criteria, visual inspection of gallery and shell demos, and final presentation/writeup coordination.
- **Michael Pham:** Mesh data structure, OpenMesh bridge, normalization, procedural test meshes, brute-force tangent-point energy, and early gradient validation infrastructure.
- **Daniel Liu:** Barnes-Hut TPE acceleration, Repulsor cross-validation, Sobolev-preconditioned descent, remeshing, gallery demo drivers, and Phase 3/4 Repulsive Shells experiments.
- **Elen Papyan:** Project website/report structure, visual assets, milestone/final presentation material, and result organization.